

Analizador Léxico C-Minus - Compiladores 2026.2

Guilherme Rozenblat e Leonardo Valladares

Repositório: <https://github.com/GuilhermeRozenblat/compiler-c-minus>

1. Linguagem escolhida:

Escolhemos C-Minus, com base no livro Compiler Construction: Principles and Practice, de Kenneth C. Louden. A gramática tem 29 regras:

1. program $\rightarrow$ declaration-list
2. declaration-list $\rightarrow$ declaration-list declaration | declaration
3. declaration $\rightarrow$ var-declaration | fun-declaration
4. var-declaration $\rightarrow$ type-specifier ID ; | type-specifier ID [NUM] ;
5. type-specifier $\rightarrow$ int | void
6. fun-declaration $\rightarrow$ type-specifier ID (params) compound-stmt
7. params $\rightarrow$ param-list | void
8. param-list $\rightarrow$ param-list , param | param
9. param $\rightarrow$ type-specifier ID | type-specifier ID []
10. compound-stmt $\rightarrow$ { local-declarations statement-list }
11. local-declarations $\rightarrow$ local-declarations var-declaration | empty
12. statement-list $\rightarrow$ statement-list statement | empty
13. statement $\rightarrow$ expression-stmt | compound-stmt | selection-stmt
| iteration-stmt | return-stmt
14. expression-stmt $\rightarrow$ expression ; | ;
15. selection-stmt $\rightarrow$ if (expression) statement
| if (expression) statement else statement
16. iteration-stmt $\rightarrow$ while (expression) statement
17. return-stmt $\rightarrow$ return ; | return expression ;
18. expression $\rightarrow$ var = expression | simple-expression
19. var $\rightarrow$ ID | ID [expression]
20. simple-expression $\rightarrow$ additive-expression relop additive-expression
| additive-expression
21. relop $\rightarrow$ <= | < | > | >= | == | !=
22. additive-expression $\rightarrow$ additive-expression addop term | term
23. addop $\rightarrow$ + | -
24. term $\rightarrow$ term mulop factor | factor
25. mulop $\rightarrow$ * | /
26. factor $\rightarrow$ (expression) | var | call | NUM
27. call $\rightarrow$ ID (args)
28. args $\rightarrow$ arg-list | empty
29. arg-list $\rightarrow$ arg-list , expression | expression

As palavras reservadas são else, if, int, return, void e while. Os símbolos são + - * / < <= > >= == != = ; , () [] { } . Um identificador segue a regra ID = letter letter*, com letter = [a-zA-Z].

Um número segue $NUM = digit\ digit^*$, com $digit = [0-9]$.

A linguagem aceita comentários entre $/^*$ e $*/$, sem aninhamento. Espaços e quebras de linha separam os tokens.

2. Scanner manual:

2.1. Subconjunto e expressões regulares:

Selecionamos as regras 4, 5, 16, 17, 18 e 21 da BNF: declaração de variável, tipo, while, return, expressão e operadores relacionais. Elas formam um subconjunto de 18 tokens.

ID: letter letter*

NUM: digit digit*

INT: int

VOID: void

WHILE: while

RETURN: return

PONTO_VIRGULA: ;

ABRE_COLCHETE: [

FECHA_COLCHETE:]

ABRE_PARENTESE: (

FECHA_PARENTESE:)

MENOR: <

MENOR_IGUAL: <=

MAIOR: >

MAIOR_IGUAL: >=

ATRIBUICAO: =

IGUAL: ==

DIFERENTE: !=

As quatro palavras reservadas também seguem a regra de ID. Por isso, o autômato usa 14 padrões; depois da leitura, uma tabela distingue essas palavras dos demais identificadores.

2.2. Construção do DFA:

Aplicamos a construção de subconjuntos. Cada estado do DFA corresponde ao fecho- ϵ de um conjunto de estados do NFA. As transições que não aparecem na tabela vão para o estado de erro.

Estado	Estados do NFA	Aceita	Transições
A	{0,1,7,13,15,17,19,21,23,25,27,29,33,37,41}	não (inicial)	letter $\rightarrow$ B, digit $\rightarrow$ C, ; $\rightarrow$ D, [$\rightarrow$ E,] $\rightarrow$ F, ($\rightarrow$ G,) $\rightarrow$ H, < $\rightarrow$ I, > $\rightarrow$ J, = $\rightarrow$ K, ! $\rightarrow$ L
B	{2,3,4,6}	ID	letter $\rightarrow$ M
C	{8,9,10,12}	NUM	digit $\rightarrow$ N

No Flex 2.6.4 usado no trabalho, input() devolve 0 ao chegar ao fim do arquivo. Verificamos esse valor para que um comentário não fechado não prenda o scanner em um laço. Nesse caso, a saída mostra ERRO: '/' na linha em que o arquivo termina. Também contamos as linhas dentro do comentário, porque os caracteres lidos por input() não passam pela regra normal de quebra de linha.

O Flex gera código lex.yy.c. O Makefile compila esse arquivo com principal.c para produzir scanner_flex. O scanner manual é compilado separadamente com o mesmo principal.c.

4. Testes e saídas:

Executamos make para compilar os scanners, make comparar para comparar as saídas do subconjunto e make completo para rodar os exemplos da linguagem completa. Cada linha da saída mostra o token e sua linha no arquivo. A tabela resume os testes:

Arquivo	Scanner	O que testa	Resultado
valido1.cm	manual e Flex	declarações com int, void, vetores e números	sem erro
valido2.cm	manual e Flex	while, os seis operadores relacionais, = e return	sem erro
valido3.cm	manual e Flex	tokens sem espaço entre eles	sem erro
invalido1.cm	manual e Flex	! isolado	ERRO: '!' na linha 2
invalido2.cm	manual e Flex	caractere #	ERRO: '#' na linha 2
invalido3.cm	manual e Flex	@ no meio de um identificador	ERRO: '@' na linha 1, entre ID val e ID r
gcd.cm	Flex	exemplo do Louden: if, else, operadores aritméticos e comentários	sem erro
ordenacao.cm	Flex	exemplo do Louden: vetores, while, if e chamadas de função	sem erro
intervalo.cm	Flex	vetor, while, if e os operadores >, >=, <= e !=	sem erro
erros.cm	Flex	#, @, \$, ! isolado e comentário não fechado	5 erros, nas linhas 2, 3, 4, 5 e 7

Nos seis testes do subconjunto, as saídas dos dois scanners foram iguais.

A palavra “válido” aqui se refere apenas à análise léxica. Por exemplo, valido2.cm contém while fora de uma função; isso não afeta o teste do scanner.

Exemplo válido do subconjunto:

valido1.cm

Código:

```
int x;  
int y;  
void z;  
int vetor[10];  
int matriz[100];
```

Saída:

```
1: palavra reservada: int  
1: ID, nome= x  
1: ;  
2: palavra reservada: int  
2: ID, nome= y  
2: ;  
3: palavra reservada: void  
3: ID, nome= z  
3: ;  
4: palavra reservada: int  
4: ID, nome= vetor  
4: [  
4: NUM, valor= 10  
4: ]  
4: ;  
5: palavra reservada: int  
5: ID, nome= matriz  
5: [  
5: NUM, valor= 100  
5: ]  
5: ;
```

Exemplo com erro no subconjunto:

invalido1.cm

Código:

```
int x;  
x = ! x;
```

Saída:

```
1: palavra reservada: int  
1: ID, nome= x  
1: ;  
2: ID, nome= x  
2: =  
2: ERRO: '!'  
2: ID, nome= x  
2: ;
```

Abaixo estão um exemplo válido e um com erros da linguagem completa. As saídas dos outros arquivos estão na pasta etapa1-analisador-lexico/saidas/.

Exemplo válido da linguagem completa:

gcd.cm

Código:

```
/* A program to perform Euclid's
   Algorithm to compute gcd. */
```

```
int gcd (int u, int v)
{ if (v == 0) return u ;
  else return gcd(v,u-u/v*v);
  /* u-u/v*v == u mod v */
}
```

```
void main(void)
{ int x; int y;
  x = input(); y = input();
  output(gcd(x,y));
}
```

Saída:

```
4: palavra reservada: int
4: ID, nome= gcd
4: (
4: palavra reservada: int
4: ID, nome= u
4: ,
4: palavra reservada: int
4: ID, nome= v
4: )
5: {
5: palavra reservada: if
5: (
5: ID, nome= v
5: ==
5: NUM, valor= 0
5: )
5: palavra reservada: return
5: ID, nome= u
5: ;
6: palavra reservada: else
6: palavra reservada: return
6: ID, nome= gcd
6: (
6: ID, nome= v
6: ,
```

```

6: ID, nome= u
6: -
6: ID, nome= u
6: /
6: ID, nome= v
6: *
6: ID, nome= v
6: )
6: ;
8: }
10: palavra reservada: void
10: ID, nome= main
10: (
10: palavra reservada: void
10: )
11: {
11: palavra reservada: int
11: ID, nome= x
11: ;
11: palavra reservada: int
11: ID, nome= y
11: ;
12: ID, nome= x
12: =
12: ID, nome= input
12: (
12: )
12: ;
12: ID, nome= y
12: =
12: ID, nome= input
12: (
12: )
12: ;
13: ID, nome= output
13: (
13: ID, nome= gcd
13: (
13: ID, nome= x
13: ,
13: ID, nome= y
13: )
13: )
13: ;
14: }

```

Exemplo com erros na linguagem completa:

erros.cm

Código:

```
int x;  
x = 10 # 3;  
x = @;  
x = 5 $ 2;  
a ! b  
/* comentario que nao fecha
```

Saída:

```
1: palavra reservada: int  
1: ID, nome= x  
1: ;  
2: ID, nome= x  
2: =  
2: NUM, valor= 10  
2: ERRO: '#'  
2: NUM, valor= 3  
2: ;  
3: ID, nome= x  
3: =  
3: ERRO: '@'  
3: ;  
4: ID, nome= x  
4: =  
4: NUM, valor= 5  
4: ERRO: '$'  
4: NUM, valor= 2  
4: ;  
5: ID, nome= a  
5: ERRO: '!'  
5: ID, nome= b  
7: ERRO: '/*'
```

5. Referencias:

LOUDEN, Kenneth C. Compiler Construction: Principles and Practice.

Lexical Analysis With Flex, for Flex 2.6.2. <https://westes.github.io/flex/manual/>